
智国学堂 TeachZero

赛项 2·识物工坊 备赛营

第 1 讲 领装备——读懂比赛，握紧工具

第 01A 课

2026 年

CCF 大模型能力认证（LMCC）课程体系

第 1 讲 领装备——读懂比赛，握紧工具



学习目标

- 说清识物工坊比赛流程：采样 → 训练 → 测试 → 迭代 → 提交 ZIP
- 会写第一个 Python 程序 (print、变量、运算)
- 掌握列表、字典与 for 循环，能批量处理数据
- 会用 if/else 做判断、用函数封装流程
- 会用 os 遍历文件夹里的图片文件

△ 注意

本节定位：这是“备赛营”第一课——先领装备，再打仗。识物工坊比赛流程（初赛窗口内自采自训自交 ZIP）是整门课的主线，本节做两件事：①把比赛规则拆明白，画出你自己的作战地图；②把 Python 最常用的部分过一遍（只学备赛用得到的），全部围绕“批量整理图片”这一个真实任务。

课前准备 (Windows 课堂环境)

- 主篇只用 Python 标准库；附篇 A（像素）需要 pillow: `python -m pip install pillow`, 验证 `python check_env.py` 中 PIL 一行 [OK]
- 教师机：Python 3.12+（安装时勾选 Add python.exe to PATH），验证：`python check_env.py`
- 课堂脚本：本讲 课堂脚本/step1 至 step7，逐个运行——先猜输出，再跑对账，最后按任务卡“试一试”改一改
- 素材：测试素材/orange（55 张），脚本自动按相对路径找到它

○、领装备：这场比赛怎么打（10 分钟）

0.1 拆解官方手册



实操环节

课堂任务：全班共读官方手册要点，各组在纸上画出比赛流程图（比谁画得全）。



识物工坊作战地图

初赛窗口内（6.15–9.20），在比赛平台 /workshop/ 完成：

1. 采样：自己拍真橙子（多光线/角度/背景）+ 难例非橙子（橘子/苹果/橙球）
2. 健康度检查：剔除重复、模糊、类别错误的图
3. 浏览器内训练：保持默认参数（迁移学习底座，动参数收益≈0）
4. 测试与迭代：新图测试 → 误判归档 → 补难例 → 重训（可多轮）
5. 提交：导出模型 ZIP（≤2MiB）上传 /workshop/competition ——以最后一次提交为准，保密测试集计分

△ 注意

三条铁律：每类 20+ 张起步；测试图绝不进训练；每天导出项目备份（数据只存在本机浏览器，机房还原卡会清掉）。

0.2 赛季积分预告

💡 备赛营怎么玩

本营共五讲，对应比赛的全流程：领装备与采样（本讲）→ 训练一·第一枪 → 训练二·照方抓药 → 训练三·冲刺与一锤定音 → 备赛准备。积分来自：采样战健康度通过率、三轮训练的迭代幅度、期末“一锤定音”模拟提交。Python 是贯穿全程的工具——现在开始领。

一、开战首胜：10 分钟练出第一个模型（30 分钟）

1.1 用现成素材先跑一遍

🔧 实操环节

操作：不写代码，先用老师备好的 测试素材/（橙子、非橙子各一小叠）体验全流程。

1. 打开训练台 → 上传两类图片（每类先给 8 张）
2. 点训练，所有参数保持默认 → 等几十秒出模型
3. 用“未见过的图”测试 5 张：对几张错几张？
4. 换一张你们组自己手机里的橙子照片再测——注意观察变化



第一仗的感受点

- 原来这么快：几十秒就训出一个模型——因为训练台的底座在千万张图上 练过“看”，你的图只是教它“认橙子”（这叫迁移学习，第 2 讲细说）
- 网图测得准，自家照片就不一定——这正是比赛的坑：保密测试集用的是真实拍摄分布，所以后面必须自己拍
- 先跑通，再学好：你已经完成了一次完整的“训练→测试”，整门课就是把 这套动作 做得更精

1.2 但注意：网图喂出来的模型上不了考场

⚠️ 注意

刚才用的素材来自网络——它帮你理解流程，不能直接拿去比赛。保密测试集是真实拍摄的照片，分布对不上分就低。从下一节开始，你们要为自己的队伍采集真正的“作战弹药”。

2.1 print 与变量

🔧 实操环节

操作：写第一个程序，用小 Python 打印一句“我要备橙子数据”。

```
# 变量：给数据起个名字
dataset = "橙子与非橙子"
print("我的任务是：准备" + dataset + "图片")
```

1. print(): 把内容打印出来
2. 变量：如 dataset，像贴标签的盒子
3. "...": 字符串，要用引号包起来

运行：python 1.py，屏幕上显示这句话。

2.2 常用运算

💡 数字与字符串基础

```
a = 10          # 整数
b = 3.5         # 小数
name = "橙子"   # 字符串
print(a + b)    # 加法 → 13.5
print(name * 3) # 重复字符串 → 橙子橙子橙子
print(f"我有{a}张{name}图片") # f字符串：把变量塞进句子
```

二、Python 速成·上：列表与循环（15 分钟）

3.1 列表：把一批图片装起来

🔧 实操环节

操作：用列表存一批图片文件名，用 for 循环逐张打印。

```
# 列表：一串数据，用[]包起来
orange_imgs = ["橙子1.jpg", "橙子2.jpg", "橙子3.jpg"]

# for循环：逐个取出
for img in orange_imgs:
    print("正在检查:", img)
```

1. 列表：[...] 存多个值，下标从 0 开始 orange_imgs[0]
2. for ... in ...: 遍历，逐个处理
3. 这正是批量看图的雏形——后面用它遍历整个文件夹

3.2 字典：给图片分类

💡 字典：键值对应

```
# 字典：名字→内容，用{}包起来
dataset = {
    "orange": ["橙子1.jpg", "橙子2.jpg"], # 橙子类
    "not_orange": ["苹果.jpg", "橘子.jpg"], # 非橙子类
}
print("橙子类有", len(dataset["orange"]), "张")
print("非橙子类有", len(dataset["not_orange"]), "张")
```

- 用 字典[键] 取值，len() 数长度
- 训练台正是按“橙子/非橙子”两类组织的

三、Python 速成·下：判断与函数（15 分钟）

4.1 if/else 判断

实操环节

操作：按数量判断数据是否达标。

```
count = 18
if count >= 20:
    print("数量充足，可以训练")
elif count >= 5:
    print("刚到最低门槛，建议再加")
else:
    print("不够5张，无法训练")
```

- if/elif/else: 按条件走不同分支
- 训练台门槛：每类至少 5 张，建议 20 张以上——这正是判断的

4.2 函数：封装成流程

def 函数

```
def count_imgs(folder):
    """统计某文件夹里jpg图片的数量"""
    # ...用一个好用的函数，把"数图"这一步封起来
    return 20

n = count_imgs("orange")
print(f"橙子类有{n}张")
```

- def 函数名(参数) 定义，return 返回
- 把常用步骤包成函数，下次直接调用

四、实操：遍历文件夹（20 分钟）

5.1 os 遍历图片

实操环节

操作：用 Python 列出某个文件夹里所有图片。

```
import os    # os 模块：跟系统文件打交道

folder = "orange"    # 换成你的橙子图片文件夹
img_count = 0
for f in os.listdir(folder):
    if f.endswith((".jpg", ".png", ".webp")):
        print("发现图片:", f)
        img_count += 1

print(f"共找到 {img_count} 张橙子图片")
```

- import os 引入系统工具
- os.listdir(folder) 列出文件夹里所有文件名
- endswith(...) 判断扩展名 (jpg/png/webp)

5.2 小任务：统计两类图片

🔧 实操环节

操作：分别统计“橙子”和“非橙子”两个文件夹的图片数，输出是否达标。

```
import os

def count_imgs(folder):
    n = 0
    for f in os.listdir(folder):
        if f.endswith((".jpg", ".png", ".webp")):
            n += 1
    return n

orange_n = count_imgs("orange")
notorange_n = count_imgs("not_orange")
print(f"橙子 {orange_n} 张, 非橙子 {notorange_n} 张")
if orange_n >= 5 and notorange_n >= 5:
    print("两类都达标, 可开始训练")
else:
    print("某类不足5张, 先补充数据")
```

1. 把“数图”写进函数 count_imgs，两边都能用
2. and：两个条件都要满足

附篇 A · 懂对手：训练台的眼睛（像素世界）

📌 学习目标

- 理解“图片在电脑里其实是数字矩阵”
- 会用 Pillow 读取、查看、保存图片
- 分清 RGB 颜色通道，看懂橙子与非橙子的颜色差异
- 理解“整图二分类”到底在分类什么
- 为识物工坊选图、看图打好认知基础

A.1 图片到底是什么

你看到的橙子照片，在电脑里是一大堆数字——每个点叫“像素”，每个像素是几个数（颜色）。

💡 图片 = 像素矩阵

一张 100×100 的图，就是 100×100 个像素点。每个像素存颜色。整张图 = 一块数字矩阵。

- 训练台分类的，就是这块“数字矩阵”的规律。
- 说“整图二分类”，就是把一整张图的数字交给模型，让它判断“这是橙子的数字规律吗”。

A.2 用 Pillow 读图

1.1 安装与读取

🔧 实操环节

操作：装 Pillow (Python 的图像库)，读一张橙子图。

```
pip install pillow
```

```
from PIL import Image # 引入 Pillow 图像库

img = Image.open("orange1.jpg") # 打开图片
print("尺寸:", img.size)        # (宽,高), 如 (300, 300)
print("格式:", img.format)       # 如 JPEG
img.show()                     # 弹窗查看
```

1. Image.open() 读图、.size 尺寸、.format 格式
2. Pillow 是处理图片最常用的 Python 库

1.2 保存与格式

💡 保存与转换

```
img.save("copy.png") # 转成 PNG 保存
img.resize((150, 150)).save("small.jpg") # 缩小后保存
```

- 识物工坊支持 JPG / PNG / WebP, 单张 $\leq 15\text{MB}$
- 用 Python 批量统一尺寸/格式, 能让数据更规整

A.3 RGB 颜色通道

2.1 像素是三个数

🔧 实操环节

操作：取图片某个位置的颜色，看它由几个数组成。

```
from PIL import Image

img = Image.open("orange1.jpg").convert("RGB")
px = img.getpixel((10, 10)) # 取左上角附近一个像素
print("该像素颜色(R,G,B):", px) # 如 (255, 150, 30)

r, g, b = px
print("红:", r, "绿:", g, "蓝:", b)
```

1. 每个像素存 3 个数：红(R)、绿(G)、蓝(B)，各自 0 255
2. 橙子像素往往 R 高、B 低（偏橙红）；这就是模型“认出橙子”的依据之一

2.2 颜色怎么看

💡 三原色混合

- (255, 0, 0) 纯红, (255, 150, 30) 橙色系, (0,0,0) 黑, (255,255,255) 白
- 橙子的特点是 红色分量高、蓝色分量低；橘子、橙色球和它接近——这正是“难例”要小心的原因

- 分得清颜色规律，才知道“非橙子”该挑哪些图（橘子、橙色球、橙色西红柿…）

A.4 整图二分类在分类什么

3.1 从像素到判断

实操环节

操作：写一个“笨办法”——用颜色比例初步猜是不是橙子（理解原理，不替代训练台）。

```
from PIL import Image

def orange_ratio(img):
    """统计偏橙色的像素占比（简化的例子）"""
    img = img.convert("RGB")
    w, h = img.size
    orange_px = 0
    for x in range(0, w, 5):
        for y in range(0, h, 5):
            r, g, b = img.getpixel((x, y))
            if r > 180 and b < 120:  # 偏橙色的粗略判断
                orange_px += 1
    total = (w // 5) * (h // 5)
    return orange_px / total

img = Image.open("orange1.jpg")
print("橙色占比:", orange_ratio(img))
```

1. 这个例子只演示“颜色统计”，真正的模型比这聪明得多
2. 但能让你理解：模型就是在找图片里的颜色/纹理规律，从而判断橙子

3.2 为什么靠颜色不够

注意

光靠颜色会误判——橘子、橙色球、橙色灯都是“红高蓝低”。所以识物工坊要收集大量、多样的真实橙子，让模型学到“橙子的纹理、形状、光泽”而不只是颜色。这也解释了为何难例（橘子/橙色物）对训练如此重要。

A.5 选图小技巧

4.1 给数据找“变化”

实操环节

取样要点（回想第1讲，用 Python 辅助整理）：

1. 橙子：不同品种、远近、大小、明暗、背景、遮挡、单个/多个
2. 非橙子：苹果、橘子、橙色球、手、桌面、空背景、其他橙色物品
3. 两类数量尽量接近；同一连拍只留几张代表图
4. 用 Python 批量查看、去掉重复、挑出难例

4.2 实操：批量看尺寸

🔧 实操环节

操作：遍历文件夹，打印每张图的尺寸，挑出异常图。

```
import os
from PIL import Image

folder = "orange"
for f in os.listdir(folder):
    if f.endswith((".jpg", ".png", ".webp")):
        img = Image.open(os.path.join(folder, f))
        print(f, img.size)
# 尺寸差异过大的，可统一 resize，让数据更规整
```

附篇 B · 数据整理工具箱

📌 学习目标

- 会用 os 遍历、批量重命名、统一图片
- 会用哈希去重，避免连拍“背答案”
- 会按比例划分训练集/测试集，防数据泄漏
- 会统计两类数量、检查数据健康度
- 明白“数据决定模型上限”，能自己备好数据

B.1 采样大作战：比赛规则

💡 这场仗怎么打

第 3 讲是整门课的核心战——比赛里 25 分的大头就在你采的图上。流程：

1. 限时采集（30 分钟）：每组拍满两类各 20 张。橙子要覆盖角度/远近/明暗/背景/遮挡；非橙子要主动加入难例（橘子/橙色球/黄柠檬）——只拍空背景得低分
2. 健康度检查（15 分钟）：用本讲 Python 工具体检自己的图，检出重复/类别错误/两类失衡——每组扣分项公开
3. 互查（10 分钟）：组间交换抽查，抓出对方漏网的问题图，抓到加你组分

积分=检查通过率 + 互查战果。这轮的图，第 5 讲模拟赛直接用。

⚠️ 注意

比赛铁律提前立：测试图绝不进训练（防泄漏）；每天导出备份（数据只存 本机浏览器）；同一连拍只留少量代表图（防“背答案”）。

B.2 批量整理图片

1.1 统一命名

🔧 实操环节

操作：把“橙子”文件夹里所有图，统一重命名为 orange_1.jpg、orange_2.jpg…

```
import os

folder = "orange"
files = [f for f in os.listdir(folder) if f.endswith((".jpg", ".png", ".webp"))]
for i, f in enumerate(sorted(files)):
    old = os.path.join(folder, f)
    new = os.path.join(folder, f"orange_{i+1}.jpg")
    os.rename(old, new)
print(f"已重命名 {len(files)} 张")
```

1. os.rename(旧, 新) 改文件名
2. 统一命名便于管理, 也避免重名覆盖

1.2 统一尺寸

💡 统一尺寸

```
from PIL import Image
import os

folder = "orange"
for f in os.listdir(folder):
    if f.endswith(".jpg"):
        img = Image.open(os.path.join(folder, f))
        img = img.resize((256, 256)) # 统一成 256×256
        img.save(os.path.join(folder, f))
```

- 尺寸统一, 数据更规整; 建议统一成正方形

B.3 去重: 别让连拍“背答案”

2.1 为什么去重

⚠ 注意

同一物体连拍几十张几乎一样的图, 模型会“背下”这张图, 而不是学会“橙子”的一般规律。训练时分数虚高, 一换图就失效。所以同一连拍只保留少量代表图。

2.2 用哈希去重

🔧 实操环节

操作: 用图片内容指纹 (哈希) 找出完全重复的图。

```
import os, hashlib

def img_hash(path):
    with open(path, "rb") as f:
        return hashlib.md5(f.read()).hexdigest()
```

```

folder = "orange"
seen = set()
for f in sorted(os.listdir(folder)):
    if not f.endswith(("jpg", "png")):
        continue
    h = img_hash(os.path.join(folder, f))
    if h in seen:
        print("重复删除: ", f)
        os.remove(os.path.join(folder, f))
    else:
        seen.add(h)

```

1. hashlib.md5 给文件算指纹，相同内容指纹相同
2. 完全相同的图删掉，保留一张代表作

2.3 相似图（进阶）

💡 相似度去重（进阶）

完全重复好去，但“几乎一样”（连拍）需算相似度。可用 Pillow 缩图后比较像素差异：

```

from PIL import Image

def is_similar(a, b, thresh=20):
    A = Image.open(a).convert("L").resize((32, 32))
    B = Image.open(b).convert("L").resize((32, 32))
    pa, pb = list(A.getdata()), list(B.getdata())
    diff = sum(abs(x - y) for x, y in zip(pa, pb)) / len(pa)
    return diff < thresh # 差异小于阈值 → 视为近似

```

convert("L") 转灰度、resize 缩图，比较两图的平均像素差。近似的只留一张。

B.4 划分训练集与测试集

3.1 为什么必须分开

⚠ 注意

防数据泄漏：绝不能用最终测试的图片去训练。若同一画面近似图同时出现在训练和测试中，自测分会虚高。要单独留一组“从未训练过”的图，专测泛化能力。

3.2 按比例划分

🔧 实操环节

操作：把橙子图按 8:2 分成训练集和测试集（测试集单独放，不进训练）。

```

import os, random, shutil

def split_folder(src, train_dir, test_dir, ratio=0.8):
    os.makedirs(train_dir, exist_ok=True)
    os.makedirs(test_dir, exist_ok=True)

```

```

files = [f for f in os.listdir(src) if f.endswith((".jpg", ".png",
".webp"))]
random.shuffle(files)
cut = int(len(files) * ratio)
for i, f in enumerate(files):
    dst = train_dir if i < cut else test_dir
    shutil.copy(os.path.join(src, f), os.path.join(dst, f))

split_folder("orange", "orange_train", "orange_test", 0.8)
split_folder("not_orange", "not_train", "not_test", 0.8)
print("划分完成：训练集80%，测试集20%")

```

1. random.shuffle 打乱, shutil.copy 复制
2. 训练集 80%、测试集 20%，两类都这样分
3. 测试集的图绝不进训练——这就是防泄漏

B.5 统计与健康度检查

4.1 统计两类数量

实操环节

操作：统计橙子/非橙子训练图数量，判断是否达标、是否平衡。

```

import os

def count(folder):
    return len([f for f in os.listdir(folder) if f.endswith((".jpg", ".png",
".webp"))])

orange_n = count("orange_train")
notorange_n = count("not_train")
print(f"训练集—橙子 {orange_n}, 非橙子 {notorange_n}")
if orange_n >= 20 and notorange_n >= 20:
    print("数量充足，且两类平衡，可训练")
elif orange_n >= 5 and notorange_n >= 5:
    print("刚过最低门槛，建议补到 20+")
else:
    print("某类不足 5 张，无法训练")

```

1. 每类 ≥ 5 才能训练，建议 20 张以上
2. 两类数量尽量接近（橙 40、非橙也约 40）

4.2 健康度检查清单

数据健康度

提交训练前，用 Python 检查：

1. 类别清晰：橙子/非橙子没混
2. 无重复、无模糊废图
3. 两类数量接近
4. 变化充分：不同角度/背景/光线，非橙子含橘子、苹果等难例

5. 训练/测试分开，无泄漏

下讲预告：弹药已备好。下一讲训练——第一枪：上传、训练、测试，把每张误判图记下来。

五、小结与作业（10 分钟）

小结

本节掌握

1. print 打印、变量存数据、f"..." 格式化
2. 列表存一串、for 循环遍历、字典按类存取
3. if/elif/else 判断、def 函数封装
4. os.listdir 遍历文件夹、endswith 过滤图片
5. 你已经亲手跑通过一次“训练→测试”——流程不再神秘

作业

回家用 Python 统计你电脑上某个图片文件夹里 jpg/png/webp 各多少张，并打印“能否训练”的判断；另外用手机拍 10 张家里的“橙色调物体”（橙子、橘子、橙球均可），下节课带来当采样战的加练素材。